Quantum Computing

Interference and Quantum Algorithms

Lecture 4 Notes

Ralph Razzouk Marc Fares Joseph Jabbour

Lebanese Science Organization

Prepared for the LebNet and LebSci Quantum Computing Course 2026

About these notes

These notes accompany a nine-lecture course on quantum computing designed and produced by the Lebanese Science Organization. Each chapter corresponds to one lecture. The lectures are concept-focused: they explain what a quantum state is, how entanglement, interference, and measurement combine to produce computation, and what is required to make that computation reliable. Linear algebra, complex numbers, and elementary probability are assumed.

Each chapter names its lecturer and date, opens with a short preface, develops its material with the derivations in place, and closes with a summary. Worked examples appear in numbered boxes, as do definitions, theorems, remarks, and common misconceptions. Works cited in the text are collected in a single reference list at the end of the document.

Conventions used throughout: qubits in a register are labelled from left to right, so that in $|ab\rangle$ the first qubit is a; the first qubit is the top wire in every circuit diagram; the computational basis of two qubits is ordered $|00\rangle, |01\rangle, |10\rangle, |11\rangle$; circuits are read from left to right, so a gate U_1 followed by a gate U_2 acts as the matrix product U_2U_1.

This chapter is exported individually from the full lecture notes (4 of 9). These notes are made available for open, public study. Reuse conditions will be stated separately by the Lebanese Science Organization.

Contents

1 Interference and Quantum Algorithms **3**
1.1 Oracles . . . 3
1.1.1 One-bit oracles . . . 4
1.1.2 Evaluation on a superposition . . . 4
1.1.3 The query model . . . 5
1.2 Phase kickback . . . 5
1.3 The Deutsch-Jozsa algorithm . . . 7
1.3.1 Deutsch's algorithm . . . 7
1.3.2 The Hadamard transform . . . 7
1.3.3 The general algorithm . . . 8
1.3.4 The three-qubit circuit . . . 9
1.3.5 Classical query complexity . . . 9
1.4 Summary . . . 11

References **12**

Chapter 1

Interference and Quantum Algorithms

Lecturer: Joseph Jabbour

Date: October 17, 2026

A quantum circuit can act on a superposition of every input at once, but a measurement returns a single outcome, so a quantum algorithm cannot simply evaluate a function everywhere and read off the results. This lecture shows how an algorithm instead arranges for the values to interfere, so that one global property of the function becomes observable with certainty while the individual values remain hidden. It introduces reversible function evaluation through an oracle, the phase kickback that converts a function value into a sign, and the Deutsch-Jozsa algorithm, the first example in the course of a provable separation in the number of queries.

1.1 Oracles

Every algorithm in this course computes something about a function $f\colon \{0,1\}^n \to \{0,1\}$. Before asking what a quantum computer can learn about f, we must settle how f is made available to it. The gates of **??** are unitary, hence reversible, and evaluation of f is in general not reversible: for $n \geqslant 2$, or whenever f is not injective, there are distinct inputs $x \neq x'$ with $f(x) = f(x')$, and no unitary can send the orthogonal states $|x\rangle$ and $|x'\rangle$ to the same state $|f(x)\rangle = |f(x')\rangle$.

The remedy is standard in reversible computing. Keep the input, and write the function value into a separate target bit by exclusive or. The map $(x, y) \mapsto (x, y \oplus f(x))$ is its own inverse, since $y \oplus f(x) \oplus f(x) = y$, and so it is a bijection on $\{0,1\}^{n+1}$ whatever f is.

Definition 1.1: Oracle for a Boolean function

Let $f\colon \{0,1\}^n \to \{0,1\}$. The oracle for f is the unitary U_f on $n+1$ qubits defined on the computational basis by

$$U_f \,|x\rangle\,|y\rangle = |x\rangle\,|y \oplus f(x)\rangle\,, \qquad x \in \{0,1\}^n, \quad y \in \{0,1\}, \tag{1.1}$$

and extended to all states by linearity. The first n qubits form the input register and the last qubit is the target.

Because U_f maps each computational basis state to a computational basis state and the map is a bijection, U_f is a permutation matrix of size 2^{n+1}. A permutation matrix has orthonormal columns, so U_f is unitary, and here $U_f^2 = \mathbb{I}$, so U_f is also its own inverse. Nothing about f has been

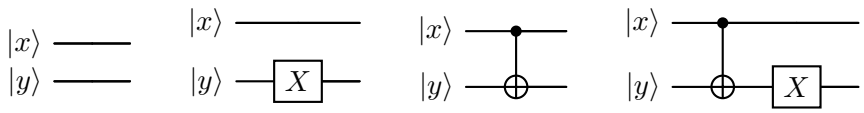
assumed: every Boolean function has an oracle. The dimension count from **??** applies, since U_f acts on a register of dimension 2^{n+1}, and the construction for small n is given in section 1.1.1.

1.1.1 One-bit oracles

For $n = 1$ the oracle acts on two qubits, the input on the top wire and the target below, in the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. The four functions and their oracles are as follows; the last column anticipates the phase kickback of section 1.2.

Function	Values	Oracle	Kickback signs on $\|0\rangle, \|1\rangle$
$f(x) = 0$	constant	$\mathbb{I} \otimes \mathbb{I}$	$+, +$
$f(x) = 1$	constant	$\mathbb{I} \otimes X$	$-, -$
$f(x) = x$	balanced	CNOT	$+, -$
$f(x) = 1 \oplus x$	balanced	$(\mathbb{I} \otimes X)\,\mathrm{CNOT}$	$-, +$

The constant function 0 leaves the target alone. The constant function 1 always flips it, which is X on the target. The identity function flips the target exactly when $x = 1$, which is CNOT controlled by the input. The function $1 \oplus x$ flips the target exactly when $x = 0$, which is CNOT followed by X on the target. As circuits, read left to right:



In each case one checks (1.1) on the four basis states. For example, $(\mathbb{I} \otimes X)\,\mathrm{CNOT}\,|1\rangle\,|0\rangle = (\mathbb{I} \otimes X)\,|1\rangle\,|1\rangle = |1\rangle\,|0\rangle$, in agreement with $y \oplus f(1) = 0 \oplus 0 = 0$ for $f(x) = 1 \oplus x$; the two flips cancel. Each of the four matrices permutes the basis states, as Definition 1.1 requires. The two constant oracles multiply every kickback state by the same sign, a global phase, while the two balanced oracles introduce a relative sign between $|0\rangle$ and $|1\rangle$; that is the whole content of Example 1.1.

For larger n, any f given by a classical circuit of AND, OR, and NOT gates can be compiled into U_f by replacing each classical gate with a reversible one (a Toffoli gate for AND, an X gate for NOT) acting on auxiliary qubits, copying the result into the target with a CNOT, and then running the reversible gates in reverse order to return the auxiliary qubits to $|0\rangle$. The last step is not optional: if the auxiliary qubits were left in a state depending on x, they would be entangled with the input register and would spoil the interference in the final $H^{\otimes n}$, in the same way that a target left in $|f(x)\rangle$ does. The number of gates in U_f is proportional to the number of gates in the classical circuit for f.

1.1.2 Evaluation on a superposition

By linearity, U_f acts on a superposition of inputs as

$$U_f \left(\sum_x \alpha_x |x\rangle \right) |0\rangle = \sum_x \alpha_x |x\rangle\,|f(x)\rangle\,. \tag{1.2}$$

A single application of U_f has produced a state in which every value $f(x)$ appears, correlated with its input. This is often described as evaluating f on all inputs at once, and the description is correct as a statement about the state. It is misleading as a statement about what we can learn.

Wrong Concept 1.1: One query lets us read every value of f

The state (1.2) does contain every value of f. Measuring it in the computational basis, however, returns a single pair $(x, f(x))$ with probability $|\alpha_x|^2$, and the state collapses to $|x\rangle\,|f(x)\rangle$. All other values are lost. This is no better than choosing an input at random and evaluating f classically. The state (1.2) is a resource, not a result: the work of an algorithm is to apply further gates so that the final measurement reports a property of the whole function rather than one entry of its table.

1.1.3 The query model

We now fix how the cost of an algorithm is counted in this chapter and the next.

Definition 1.2: Query and query complexity

A query is one application of the oracle U_f, on whatever state the algorithm has prepared. The query complexity of an algorithm is the number of queries it makes. A classical query is an evaluation of f at one chosen input. The classical query complexity of a problem is the smallest number of classical queries that suffices for every function allowed by the problem.

The query model treats f as a black box: the algorithm may apply U_f but may not inspect how U_f is built. Two consequences deserve to be stated at the outset.

First, the number of queries is not the running time. Given a classical circuit that computes f with g elementary gates, one can build U_f from a comparable number of reversible gates, together with auxiliary qubits that are returned to $|0\rangle$ after use. The construction is not free, and its cost is the same order as a classical evaluation of f. A query count therefore compares the number of times f must be consulted, not the number of gates executed.

Second, a lower bound on classical queries assumes black-box access. If a classical algorithm is allowed to read the circuit for f, it may exploit structure that the query model forbids. Separations proved in the query model are exact and rigorous within that model, and they are statements about that model.

Remark 1.1 The oracle model

The oracle model isolates one question: how many times must f be consulted to learn a given property of it? This question can be answered exactly for both classical and quantum algorithms, which is why the first quantum algorithms were formulated in it. **??** uses the same oracle to formulate unstructured search, and the period-finding routine of **??** is also, at its core, a query algorithm. The model is a starting point, not the whole story, and we will state its limits each time a comparison is made.

1.2 Phase kickback

The oracle (1.1) writes $f(x)$ into the target as a bit. Interference, as **??** established, acts on relative phases, not on bits. The device that converts one into the other is to prepare the target not in $|0\rangle$ but in $|-\rangle$.

Proposition 1.1 Phase kickback

For every $x \in \{0,1\}^n$,

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle . \tag{1.3}$$

Consequently, for any state $\sum_x \alpha_x |x\rangle$ of the input register,

$$U_f \left(\sum_x \alpha_x |x\rangle \right) |-\rangle = \left(\sum_x (-1)^{f(x)} \alpha_x |x\rangle \right) |-\rangle . \tag{1.4}$$

Proof. With $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ and (1.1),

$$U_f |x\rangle |-\rangle = \frac{1}{\sqrt{2}} \left(U_f |x\rangle |0\rangle - U_f |x\rangle |1\rangle \right) = |x\rangle \otimes \frac{1}{\sqrt{2}} \left(|f(x)\rangle - |1 \oplus f(x)\rangle \right) . \tag{1.5}$$

If $f(x) = 0$ the second factor is $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$. If $f(x) = 1$ it is $\frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = -|-\rangle$. In both cases the result is $(-1)^{f(x)} |x\rangle |-\rangle$, which is (1.3). Equation (1.4) follows by linearity, since U_f acts on each term $\alpha_x |x\rangle |-\rangle$ separately and the common factor $|-\rangle$ can be taken outside the sum. The target is in a product state with the input register both before and after the query, and so it carries no information about f; all of it is in the signs of the input register. ■

The same calculation with the target in $|0\rangle$ instead of $|-\rangle$ produces (1.2), in which the input register and the target are entangled whenever the amplitudes α_x are nonzero on inputs with both function values, and in particular for the uniform superposition whenever f is not constant. The choice of $|-\rangle$ is what prevents this entanglement and keeps the input register in a pure superposition on which $H^{\otimes n}$ can act.

Three features of (1.4) carry the rest of the chapter. The target qubit is unchanged; it enters as $|-\rangle$ and leaves as $|-\rangle$, unentangled with the input register, and may be discarded. The value $f(x)$ has moved from the target into the input register, where it appears as a sign. And for a single basis state $|x\rangle$ that sign is a global phase, unobservable by **??**; it becomes observable only when the input register is in a superposition, where different x acquire different signs and the signs are relative phases. The function values have been kicked back from the target to the input, which is why the effect is called phase kickback.

Remark 1.2 Kickback as a controlled operation

For $n = 1$ and $f(x) = x$ the oracle is CNOT with the first qubit as control. Equation (1.3) then reads CNOT $|x\rangle |-\rangle = (-1)^x |x\rangle |-\rangle$: with the target in $|-\rangle$, CNOT acts as Z on its control. This is the identity $HZH = X$ of **??** seen on two qubits. Conjugating the target of CNOT by H produces the controlled-Z gate of **??**, whose matrix is diagonal with entries $1, 1, 1, -1$ and is symmetric between the two qubits, so that control and target are not distinguishable. Phase kickback is the general form of this observation: a controlled operation whose target is in an eigenvector of the operation deposits the eigenvalue as a phase on the control. Here the eigenvector is $|-\rangle$, the operation is X, and the eigenvalue is -1. **??** uses the same mechanism with eigenvalues $e^{i\theta}$ in place of -1.

We now combine kickback with the uniform superposition. Applying $H^{\otimes n}$ to $|0\rangle^{\otimes n}$ gives $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle$, the equal superposition of all inputs. One query with the target in $|-\rangle$ turns it into

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle . \tag{1.6}$$

The entire table of f is encoded in the pattern of signs of (1.6), and a computational-basis measurement of this state is useless: every outcome has probability 2^{-n} regardless of f. In **??** the sign between two amplitudes was made visible by a Hadamard gate before measurement. The same idea, applied to n qubits, is the final step of the algorithm in the next section.

1.3 The Deutsch-Jozsa algorithm

The Deutsch-Jozsa problem [1] is a promise problem: the function is guaranteed in advance to belong to one of two classes, and the task is to decide which.

Definition 1.3: Constant and balanced functions

A function $f\colon \{0,1\}^n \to \{0,1\}$ is constant if it takes the same value on every input, and balanced if it takes the value 0 on exactly 2^{n-1} inputs and the value 1 on the remaining 2^{n-1}. The Deutsch-Jozsa problem is: given oracle access to an f that is promised to be either constant or balanced, decide which.

1.3.1 Deutsch's algorithm

For $n = 1$ there are four functions. The two constant ones satisfy $f(0) = f(1)$; the two balanced ones satisfy $f(0) \neq f(1)$. Deciding classically requires both values, hence two queries. The one-qubit algorithm, due to Deutsch [2] in the deterministic form later given by Cleve, Ekert, Macchiavello, and Mosca [3], applies H to $|0\rangle$, queries with the target in $|-\rangle$, applies H again, and measures. After the query the input qubit is $\frac{1}{\sqrt{2}}\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right)$ by (1.6).

Example 1.1 (Interference at $|0\rangle$)

If f is constant, the two signs agree and the input qubit is $\pm|+\rangle$; the final H sends it to $\pm|0\rangle$ and the measurement gives 0 with certainty. If f is balanced, the signs differ and the input qubit is $\pm|-\rangle$; the final H sends it to $\pm|1\rangle$ and the measurement gives 1 with certainty. The amplitude of $|0\rangle$ after the last gate is $\frac{1}{2}\left((-1)^{f(0)} + (-1)^{f(1)}\right)$: the two contributions add in the constant case and cancel in the balanced case. One query decides the problem, and the two circuits are exactly the circuits HH and HZH of **??**.

1.3.2 The Hadamard transform

For n input qubits the single Hadamard gates of Example 1.1 are replaced by the Hadamard transform $H^{\otimes n}$. The decisive fact about $H^{\otimes n}$ is its action on a basis state,

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} (-1)^{x\cdot y}|y\rangle, \qquad x\cdot y = x_1y_1 \oplus \cdots \oplus x_ny_n, \tag{1.7}$$

which is derived as follows. For one qubit, $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, which can be written in one formula as

$$H|x_i\rangle = \frac{1}{\sqrt{2}} \sum_{y_i\in\{0,1\}} (-1)^{x_iy_i}|y_i\rangle, \qquad x_i \in \{0,1\}, \tag{1.8}$$

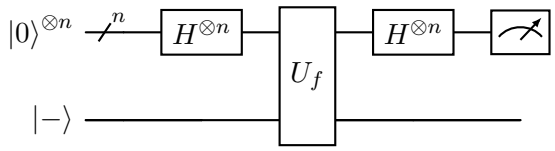
since $(-1)^{x_iy_i}$ equals -1 exactly when $x_i = y_i = 1$. For $x = x_1x_2\cdots x_n$ the state $|x\rangle$ is the tensor product $|x_1\rangle \otimes \cdots \otimes |x_n\rangle$ and $H^{\otimes n}$ acts factor by factor, so

$$H^{\otimes n}|x\rangle = \bigotimes_{i=1}^{n}\left(\frac{1}{\sqrt{2}}\sum_{y_i}(-1)^{x_iy_i}|y_i\rangle\right) = \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} (-1)^{x_1y_1+\cdots+x_ny_n}|y\rangle. \tag{1.9}$$

Since only the parity of the exponent matters, the exponent may be replaced by $x \cdot y = x_1 y_1 \oplus \cdots \oplus x_n y_n$, the bitwise inner product modulo 2, which is (1.7). Two special cases are used in this chapter. For $x = 0 \cdots 0$ every sign is $+1$ and $H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_y |y\rangle$ is the uniform superposition. For $y = 0 \cdots 0$ the coefficient of $|0 \cdots 0\rangle$ in $H^{\otimes n} |x\rangle$ is $1/\sqrt{2^n}$ for every x. Since $H^2 = \mathbb{I}$, also $(H^{\otimes n})^2 = \mathbb{I}$, so $H^{\otimes n}$ is its own inverse.

1.3.3 The general algorithm

The general algorithm replaces the single Hadamard gates by $H^{\otimes n}$ and is otherwise identical.



By (1.7), every basis state $|x\rangle$ contributes to $|0 \cdots 0\rangle$ with the same amplitude $1/\sqrt{2^n}$, because $x \cdot 0 = 0$ for every x. Applying $H^{\otimes n}$ to (1.6) and collecting the coefficient of $|0 \cdots 0\rangle$ therefore gives the following statement.

Theorem 1.1 Deutsch-Jozsa

Apply $H^{\otimes n}$ to the input register $|0\rangle^{\otimes n}$, query U_f once with the target in $|-\rangle$, apply $H^{\otimes n}$ to the input register again, and measure it in the computational basis. The amplitude of the outcome $0 \cdots 0$ is

$$a_0 = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)}. \tag{1.10}$$

If f is constant then $a_0 = \pm 1$ and the outcome is $0 \cdots 0$ with probability 1. If f is balanced then $a_0 = 0$ and the outcome is never $0 \cdots 0$. Under the promise, one query decides the problem with certainty.

Proof. Follow the circuit stage by stage, writing only the input register once the target has factored out.

$$|0\rangle^{\otimes n} |-\rangle \xrightarrow{H^{\otimes n} \otimes \mathbb{I}} \frac{1}{\sqrt{2^n}} \sum_x |x\rangle |-\rangle \tag{1.11}$$

$$\xrightarrow{U_f} \frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle |-\rangle \tag{1.12}$$

$$\xrightarrow{H^{\otimes n} \otimes \mathbb{I}} \frac{1}{2^n} \sum_x (-1)^{f(x)} \sum_y (-1)^{x \cdot y} |y\rangle |-\rangle \tag{1.13}$$

$$= \frac{1}{2^n} \sum_y \left(\sum_x (-1)^{f(x) \oplus x \cdot y} \right) |y\rangle |-\rangle . \tag{1.14}$$

The amplitude of $|y\rangle$ in the input register is $a_y = 2^{-n} \sum_x (-1)^{f(x) \oplus x \cdot y}$. Setting $y = 0 \cdots 0$ makes $x \cdot y = 0$ for every x and gives (1.10). If f is constant with value c, every term equals $(-1)^c$ and $a_0 = (-1)^c$, so $|a_0|^2 = 1$ and, by normalization, every other a_y vanishes. If f is balanced, exactly 2^{n-1} terms equal $+1$ and 2^{n-1} equal -1, so $a_0 = 0$ and the probability of the outcome $0 \cdots 0$ is zero; the remaining amplitudes a_y for $y \neq 0 \cdots 0$ depend on f and have squared moduli summing to 1 because $H^{\otimes n}$ is unitary. For a function that is neither, the sum in (1.10) lies strictly between -2^n and 2^n and is nonzero, which is the content of Remark 1.3. ∎

The two cases follow because a constant function contributes 2^n equal signs and a balanced function contributes 2^{n-1} signs of each kind. What the algorithm has done is to arrange that the 2^n amplitudes reaching $|0\cdots0\rangle$ interfere constructively when f is constant and cancel exactly when f is balanced. It has not read any value of f. The final state in the balanced case depends on f, and its measurement gives some nonzero string y whose distribution depends on f, but the algorithm only asks whether y is zero.

For $n = 1$ the calculation reduces to

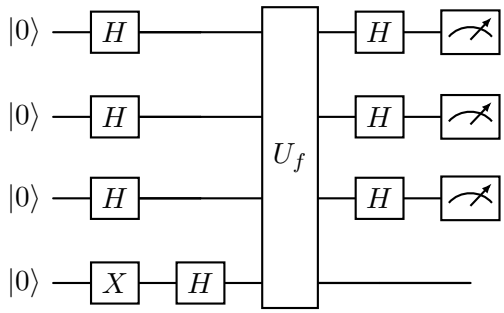
$$a_0 = \tfrac{1}{2}\left((-1)^{f(0)} + (-1)^{f(1)}\right), \qquad a_1 = \tfrac{1}{2}\left((-1)^{f(0)} - (-1)^{f(1)}\right), \tag{1.15}$$

which is the content of Example 1.1.

> **Remark 1.3** The promise is essential
>
> For a function that is neither constant nor balanced, (1.10) gives an a_0 strictly between -1 and 1 and different from 0, so the outcome $0\cdots0$ occurs with a probability that is neither 0 nor 1. The algorithm then returns an answer that means nothing. The certainty in Theorem 1.1 is a joint property of the circuit and the promise. Removing the promise changes the problem. Deciding whether an arbitrary f is constant requires detecting a function that differs from a constant at a single input, which means finding that input; this is the search problem of **??**, and its quantum query complexity is still exponential in n.

1.3.4 The three-qubit circuit

The bundle notation of section 1.3.3 stands for the following circuit, drawn for three input qubits. The target is prepared in $|-\rangle$ by applying X and then H to $|0\rangle$.



The three meters return the string $y_1y_2y_3$. Under the promise, $y = 000$ with certainty if f is constant and $y \neq 000$ with certainty if f is balanced. The target is not measured; its state is $|-\rangle$ throughout and carries no information.

1.3.5 Classical query complexity

We now make the comparison with classical algorithms, and make it carefully.

A deterministic classical algorithm that is required to be correct on every constant or balanced function needs $2^{n-1} + 1$ queries in the worst case. Having seen 2^{n-1} equal values it still cannot decide, because the remaining 2^{n-1} values may all agree with them or all differ from them, and both completions satisfy the promise. One more query settles the matter. Against this, Theorem 1.1 uses one query. In the exact, deterministic setting, the separation is from $2^{n-1} + 1$ to 1, and it is exponential in n.

A randomized classical algorithm, however, solves the problem with a handful of queries if a small error probability is tolerated. Query f at k independently and uniformly chosen inputs. If

any two values differ, answer balanced; this is never wrong. If all k agree, answer constant; this is wrong only if f is balanced and all k samples happened to land on inputs with the same value, which has probability 2^{1-k}. Eleven queries give an error below one in a thousand, for every n.

Both counts are verified as follows.

Deterministic, exact. Suppose an algorithm has queried $k \leqslant 2^{n-1}$ distinct inputs and all values agree, say all equal 0. There remain $2^n - k \geqslant 2^{n-1}$ unqueried inputs. Setting all of them to 0 gives a constant function consistent with everything seen. Setting exactly 2^{n-1} of them to 1 and the rest to 0, which is possible since at least 2^{n-1} remain, gives a balanced function consistent with everything seen. The algorithm cannot distinguish these two functions, so it cannot answer correctly on both. Hence $2^{n-1}+1$ queries are necessary in the worst case. They are also sufficient: if any two of the queried values differ the function is not constant, so it is balanced; if $2^{n-1}+1$ values agree the function cannot be balanced, so it is constant. The worst-case deterministic classical query complexity is exactly $2^{n-1}+1$.

Randomized, bounded error. Query f at k inputs chosen independently and uniformly at random. If the function is constant, all values agree and the answer constant is correct with certainty. If the function is balanced, each value is 0 or 1 with probability $\frac{1}{2}$ independently, so all k agree with probability $2 \cdot 2^{-k} = 2^{1-k}$, and only then is the wrong answer given. The error probability is at most 2^{1-k} independently of n; for $k = 11$ it is below 10^{-3}. This is why the exponential separation of the previous paragraph is a statement about exact algorithms only.

Wrong Concept 1.2: Deutsch-Jozsa proves an exponential speedup

The exponential gap is between one exact quantum query and $2^{n-1}+1$ exact deterministic classical queries. Against a randomized classical algorithm with bounded error, the gap is between one query and a constant number of queries, which is no asymptotic advantage at all. Deutsch-Jozsa is a separation between exact quantum and exact classical computation in the query model. It is not evidence that quantum computers solve problems exponentially faster than classical randomized computers, and it says nothing about running time outside the oracle model. Its value is that it exhibits, in the simplest possible setting, the mechanism that later algorithms use: superpose the inputs, encode the function in phases, and interfere.

Remark 1.4 The three-stage pattern

The circuit of Theorem 1.1 has three stages: a transformation that spreads amplitude over all inputs, a query that writes the function into relative phases, and a transformation that concentrates amplitude according to a global property of those phases. In **??** the concentration step is repeated many times and becomes amplitude amplification, and the oracle is used exactly in the phase form (1.4). In **??** the Hadamard transform $H^{\otimes n}$ is replaced by the quantum Fourier transform, which detects periodicity in the phase pattern rather than constancy, and that is what makes period finding, and hence factoring, possible. Bernstein-Vazirani and Simon's algorithm sit between these two and are omitted from this course.

1.4 Summary

A Boolean function is made available to a quantum computer as the reversible oracle $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$, a permutation matrix that keeps the input because erasing it would not be unitary. A query is one use of U_f; the query count is not the gate count, and the oracle model treats f as a black box. Applying U_f to a superposition of inputs produces every value of f in the state but lets us read only one of them. Preparing the target in $|-\rangle$ kicks the value $f(x)$ back into the input register as the sign $(-1)^{f(x)}$, converting bits into relative phases; this is a controlled operation acting on an eigenvector of its target. The Deutsch-Jozsa algorithm spreads amplitude with $H^{\otimes n}$, queries once, and applies $H^{\otimes n}$ again, so that the amplitude of $|0 \cdots 0\rangle$ equals $2^{-n} \sum_x (-1)^{f(x)}$, which is ± 1 for a constant function and 0 for a balanced one. The certainty depends on the promise. The separation is one query against $2^{n-1} + 1$ exact deterministic classical queries; a randomized classical algorithm needs only a constant number of queries for bounded error, so the algorithm demonstrates a mechanism rather than a general speedup.

Homework 2, part B, can be attempted after this lecture.

References

[1] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society A*, 439:553–558, 1992.

[2] David Deutsch. Quantum theory, the Church-Turing principle and the universal quantum computer. *Proceedings of the Royal Society A*, 400:97–117, 1985.

[3] Richard Cleve, Artur Ekert, Chiara Macchiavello, and Michele Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society A*, 454:339–354, 1998.